



نظري

5



20



3000 sp

كلية الهندسة المعلوماتية

السنة الثانية

التعليمات المنطقية ، الإجراءات ،
 وأنماط العنوان في معمارية
 MIPS

د. خولة العلي



RB Informatcs; 10/06/2026

بنیان الحاسوب ا

بسم الله الرحمن الرحيم

العمليات المنطقية

بدايةً سنتحدث عن بعض تعليمات إزاحة منطقية

1- تعليمة SLL (Shift Left Logical) : إزاحة منطقية لليسار

تذكرة بنوع R-Type : تتألف من 32 بت و تحجز في الذاكرة

op	rs	rt	rd	shift	func
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

sll rd, rt ,shmat

صيغة كتابة التعليمة :

حيث shmat هي اختصار ل shift amount
 تأخذ 5 bits

sll \$t2 , \$s0 , 4

الصيغة السابقة مكافئة لـ

mul \$t2 , \$s0 , 16

$\$t2 = \$s0 * 2^4$

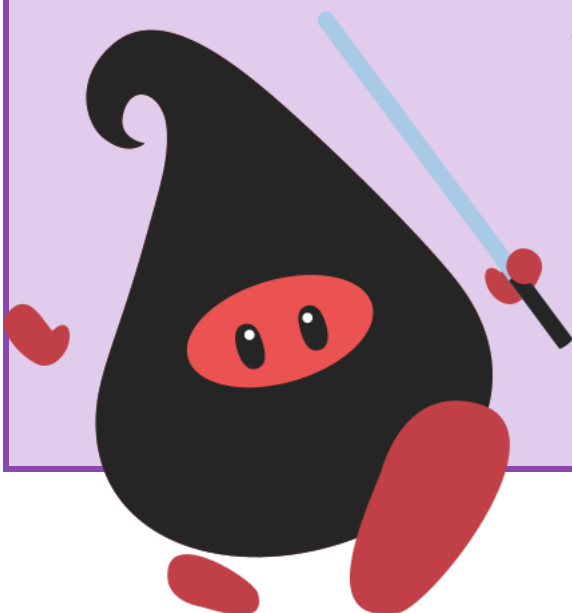
$\$t2 = \$s0 * 16$

وبالتالي نستنتج أن هذه التعليمة مقدار انزياحها يعادل i

sll \$t2, \$s0 , i

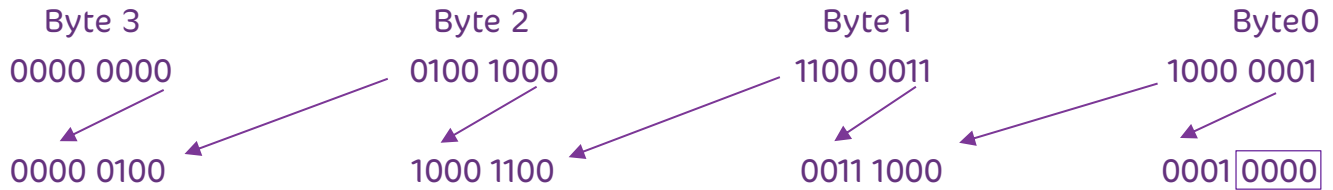
حيث i هي :

sll rd , rt , i



مثال:

عند الانزياح إلى اليسار العدد يصبح أكبر
كما في المثال:



2- التعليمة Srl (Shift Right Logical) إزاحة منطقية نحو اليمين عند الإنزياح إلى أن يصبح العدد أصغر

Srl rd, rt, shmat

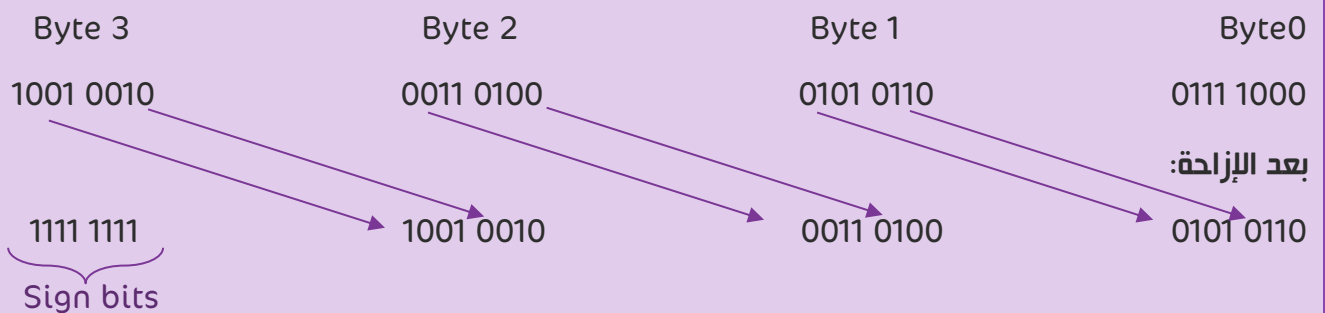
صيغة التعليمة:

مقدار الإنزياح يعادل القسمة على 2^i
يقوم بعمل التعليمة srl لكن نحو اليمين

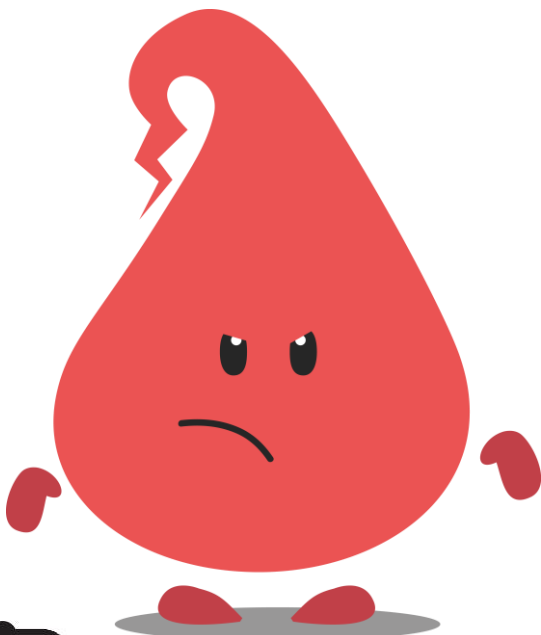
3- التعليمة Sra (Shift right arithmetic) إزاحة حسابية نحو اليمين

هذه التعليمة تملأ الفراغات بعد الإزاحة بـ sign bits (بتات الإشارة) البت 31

مثال: (الإزاحة نحو اليمين تجعل العدد أصغر)



ملاحظة: الإزاحة هنا كانت 8-bits



God will make you
rich !!!!

Stop crying for
those losers !!!

العمليات المنطقية -2

1. التي من نوع R-Type:

التعليمات من هذا النوع يكون الـ $op=0$ ويحدد المعالج ماهية العملية حسب رقم الـ $func$ 1- تعليمة AND ($op=0$, $func=36$)

مثال:

```
and $t0 , $t2 , $t1
#and rd , rs , rt
```

فائدة أرسيزية:

-تستخدم تعليمة and لعزل البتات (Bit Masking) حيث تسمح لنا بعزل أو استخراج بتات معينة كأن نأخذ البتات الأعلى من عدد مكون من 8 bits ونعلم أن (أي عدد and 0) هو 0 ... إذا مكنا عمل clear للعدد ونعلم أن (أي عدد and 1) هو العدد نفسه

مثال: لدينا في المسجل $t1$ الرقم التالي 1010 0101 ونريد فقط البتات الأربع العلوية ... ما التعليمة المناسبة؟ واحفظ النتيجة في $t0$

الحل:

```
addi $t2 , $zero , 0xF0
and $t0 , $t1 , $t2
```

• AND ($op = 0$, $func = 36$)

op	rs	rt	rd	shamt	func
000000	01010	01001	01000	00000	100100

Useful for masking bits

```
and $t0 , $t2 , $t1
```

2- التعليمة or ($op=0$, $func=37$)

مثال:

```
or $t0 , $t2 , $t1
#or rd , rs , rt
```

فائدة أرسيزية:

-تستخدم تعليمة or لدمج حقول البتات أي دمج مجموعتين من سجلين مختلفين combining في سجل واحد أو لدمج أو لتفعيل بتات معينة بجعل قيمتها 1 الزاميا مع الحفاظ على باقي قيم البتات الأصلية دون تغيير

تمرين:

لدينا العدد 0000 0110 في المسجل $t2$ نريد تغيير قيمة البت الرابع حتى يصبح العدد في النظام العشري يساوي 14 وفي النهاية نريد حفظ العدد في المسجل $t0$

الحل:

```
addi $t1 , $zero , 0x08
or $t0 , $t2 , $t1
```

- OR (op = 0, funct = 37)

op	rs	rt	rd	shamt	funct	
000000	01010	01001	01000	00000	100101	Useful for combining bit field

or \$t0, \$t2, \$t1

3- تعليمة NOR (op=0 , func=39)

مثال 1:

nor \$t0 , \$t2 , \$t1

مثال 2:

nor \$t0 , \$zero , \$t1

كمان فائدة أرسيزية:

تستخدم تعليمة nor بشكل أساسي في عكس البتات وذلك نظراً لعدم وجود تعليمة NOT من بين تعليمات MIPS إضافة أيضاً لعدم وجود تعليمة nori في MIPS

تمرين: لدينا العدد 1010 1010 في المسجل \$t0 ونريد حفظ معاكسه في المسجل \$t2 استخدم تعليمات مناسبة من تعليمات MIPS لقلب العدد وحفظه في المسجل المطلوب:

الحل:

nor \$t2 , \$zero , \$t0

للحصول على معاكس عدد باستخدام nor نقوم بالتالي

$$\overline{(a \text{ or } 0)} = a \text{ nor } 0 = (\bar{a}) = \text{not } a$$

وكما نعلم أن القيمة 0 محفوظة في المسجل \$zero

- NOR (op = 0, funct = 39)

op	rs	rt	rd	shamt	funct	
000000	01010	01001	01000	00000	100111	Useful for inverting bits

nor \$t0, \$t2, \$t1

2. التي من النوع I-Type :

نلاحظ أن op هو المختلف وبالتالي المعالج يعرف نوع التعليمة من نوع | حسب قيمة op

- Recall Immediate format

op	rs	rt	constant or address
6 bits	5 bits	5 bits	16 bits

1- (op=12) andi :

استخداماتها نفس استخدامات and لكن بما أنها من نوع I-Type فهي register to immediate

andi \$t1 , \$t0 , 0x0F

مثال:

حيث :

\$t0 تحتوي 0000 1000 تصبح النتيجة مخزنة في \$t1 هي 0000 1000

- AND Immediate (op = 12)

op	rs	rt	constant
001100	01001	01000	0000000011111111

andi \$t0, \$t1, 255

ori (op=13) -2 :

استخداماتها نفس استخدامات or ولكن بما أنها من نوع I-Type فهي register to immediate
 مثال: ori \$t1, \$t0, 0x0F

بفرض \$t0 تحتوي 0011 0000 تصبح النتيجة المخزنة في \$t1 هي 0011 1111

• OR Immediate(op = 13)

	op	rs	rt	constant
ori \$t0, \$t1, 255	001101	01001	01000	0000000011111111

Branching التفرع

التفرع ينفذ التعليمات بغير الترتيب المكتوب به البرنامج أي Out Of Sequence
 يوجد لدينا نوعان من التفرع:

1- Conditional "التفرع الشرطي":

من التعليمتان: -1 beq (branch if equal)

-2 bne (branch not equal)

Op	rs	rt	Constant or address
----	----	----	---------------------

أي بمجرد قراءة المعالج لأول 6 بتات في حال كانت مساوية ل 4 يكتشف أنها beq أما 5 فهي bne

2- Unconditional "التفرع غير الشرطي":

من التعليمتان: -1 j (jump)

-2 jr (jump register)

-3 jal (jump and link)

Conditional branch "الفروع الشرطية":

هذا النوع من التعليمات يقوم بالانتقال (التفرع) إلى القسم المسمى إذا كان الشرط محققاً غير ذلك فهو يكمل تنفيذ التعليمات بالترتيب

مثال:

لدينا التعليمة الشرطية التالية : L1, \$s3, \$s2 beq والتي تعني نه إذا كانت if(\$s2==\$s3) انتقل إلى التعليمة المسماة L1 غير ذك أكمل التنفيذ بشكل تسلسلي

لدينا opcode=4 للتعليمة beq بينما opcode=5 للتعليمة bne

مثال:

bne \$s2, \$s3, L1

وهذه التعليمة تعني إذا كان $s2 \neq s3$ انتقل إلى التعليمات المسماة L1

UnCoditional brach "الفروع غير الشرطية":

هذا النوع من التعليمات لا يحتاج شرط لكي يتنفذ يكتفي فقط بتوجيهه إلى اسم القسم الذي نريده تنفيذه مثلاً
J L1

أي انتقل إلى block of code اللي اسمه L1

التعليمة ال هي تعليمة من نوع جديد من أنواع تعليمات MIPS والذي يدعى J-Type حيث تتوزع ال 32-bit فيه كما يلي:

op	address
6-bit	26-bits

Basic Blocks (الكتل الأساسية)

ملاحظة:

الكتل الاساسية لها موجه دخول واحد للدخول من البداية وموجه واحد للخروج من النهاية وبمجرد بدء تنفيذ اول تعليمة فيها ضمن المعالج تنفيذ باقي تعليمات الكتلة دون مقاطعة او قفز

الكتلة الأساسية هي عبارة عن تتابع عدة تعليمات بدون:

1- أي فروع مغروسة بداخلها (مثل تعليمة jump) عدا في النهاية

2- أي فروع من البرنامج يكون هدفها تعليمة معينة موجودة في منتصف الكتلة (عدا في البداية)

وبالتالي لدينا ال compiler يقوم بتنفيذ جميع تعليمات الكتلة الأساسية بالتتابع من البداية إلى النهاية

Conditional Branches : Compiling if statements

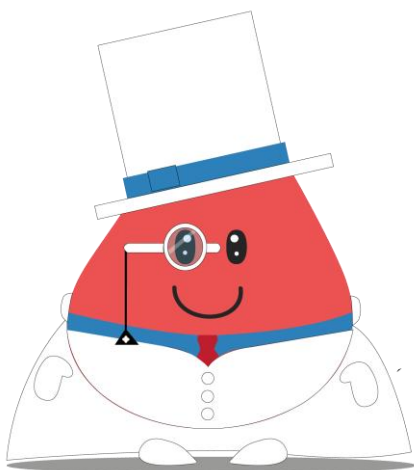
لدينا في لغة C الكود التالي:

```
if(i==j)
    f=g+h;
else
    f=g-h;
```

المتغيرات هي z , i , h , g , f بقابلهم أسماء المسجلات على الترتيب \$s4 -> \$s0

الـ code المقابل بلغة التجميع MIPS :

```
bne $s3, $s4, Else
add $s0, $s1, $s2
J Exit
```



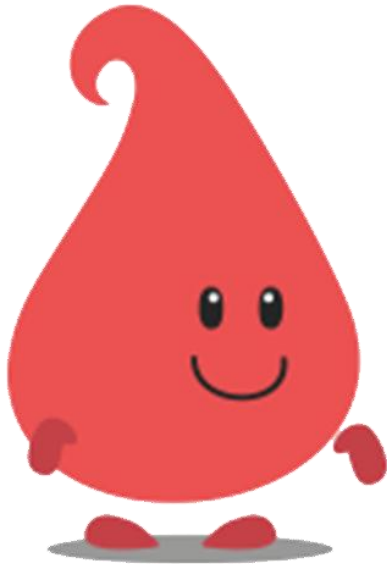
Else: sub \$s0, \$s1, \$s2

Exit:

نلاحظ أننا استخدمنا تعليمة bne في MIPS لتقوم بنفس عمل التعليمة if التي نعرفها في لغات البرمجة عالية المستوى.

Conditional Branches: Compiling Loop Statements

الحلقات أيضاً تعتمد على التعليمات الشرطية :
لدينا الحلقة التالية :



```
While (save [i]=k){
    i+=1;}
```

في MIPS لدينا $i = \$s3$ و $k = \$s5$ و $\text{save}[0] = \$s6$ إذاً يقابل هذه الحلقة بلغة MIPS

```
Loop : sll $t1, $s3, 2
      add $t1, $t1, $s6
      lw $t0, 0($t1)
      bne $t0, $s5, Exit
      addi $s3, $s3, 1
      j loop
Exit:
```

More Conditional Operation

لدينا التعليمة الجديدة **slt** (set if less than) ($op=10$) والتي تقوم بمقارنة قيمتين وبدلاً من الانتقال لمكان تعليمات آخر كما في التعليمتين beq, bne تقوم تعليمة slt بحفظ القيمة إما 0 أو 1 في المسجل الهدف الصيغة: $\text{slt } rd, rs, rt$ إذا $rs < rt$ يقوم الـ Compiler بحفظ القيمة 1 في rd عدا ذلك يقوم بحفظ 0 تعليمة أخرى من نوع I-Type تقوم بنفس عمل slt وهي **slti** أي: $\text{slti } \$t0, \$s3, 200$ أي أنه إذا $\$s3 < 200$ كن أسند القيمة 1 لـ $\$t0$ وإلا $\$t0 = 0$

استخدامات التعليمة slt :

مثلاً لدينا حلقة for($\text{int } i=10; i \geq 0; i--$): فيها تعليمة شرط الوقوف , وبما أن beq, bne وحدهما لا تكفيان للحل , وبفرض أن $i = \$t1$ إذاً:

```
slt $t0, $t1, $zero
bne $t0, $zero, exit
```

Branch Instruction Design (تصميم التعليمات الفرعية)

الـ MIPS Compilers تستخدم تعليمات bne, beq, slt, slti مع قيم ثابتة من الـ 0 (\$zero) لتحصل على كل أنواع المقارنات المنطقية ($=, !=, <, >, \leq, \geq$) لماذا لا تستخدم التعليمات bge, blt لتحقيق المقارنات السابقة ؟

لأن التعليمات `blt(branch less than)` , `bge(branch greater than)` هي تعليمات معقدة جدا وتحتاج لتنفيذها دورات ساعة أطول ... مما يجعل زمن تنفيذها أطول

المقارنة بين signed vs unsigned comparison

- 1- مقارنة الأرقام مع إشارة signed باستخدام التعليمات `slt` , `slti`
- 2- مقارنة الأرقام دون إشارة unsigned باستخدام التعليمات `sltu` , `sltiu`

مثال : لدينا المسجلين الآتيين يحويان القيم التالية :

`$s0=1111 1111 1111 1111 1111 1111 1111 1111`

`$s1=0000 0000 0000 0000 0000 0000 0000 0001`

ماذا ستكون القيم المحتواة في المسجلين `$t0` , `$t1` حسب الكود:

`slt $t0 , $s0 , $s1 #signed`

`+1 > -1 , $t0 = 1`

`sltu $t1 , $s0 , $s1 #unsigned`

`+1 < 4,294,967,295`

`$t1=0`

نلاحظ أن الـ compiler قام بقراءة الأرقام المحفوظة في المسجلين `$s0` , `$s1` حسب التعليمات المستخدمة قبلها

Example to check for Array bounds

يحدث إرور عندما يحاول المبرمجون الوصول لعنصر خارج حدود المصفوفة حيث يتم تفقد إذا كان العنصر خارج المصفوفة من خلال المقارنة التالية : $0 \leq x < y$ حيث x هي العنصر و y هو حدود المصفوفة -> العناصر خارج المصفوفة تكون أكبر أو تساوي حجم المصفوفة أو تكون سالبة ???

كود:

`Sltu $t0 , $s1 , $t2`

`Beq $t0 , $zero , IndexOutOfBounds`

MIPS Vocabulary So Far

Name	Format	Example						Comments
<code>nor</code>	R	0	18	19	17	0	39	<code>nor \$s1, \$s2, \$s3</code>
<code>sll</code>	R	0	0	19	17	4	0	<code>sll \$s1, \$s2, 4</code>
<code>srl</code>	R	0	0	19	17	4	2	<code>srl \$s1, \$s2, 4</code>
<code>slt</code>	R	0	18	19	17	0	42	<code>slt \$s1, \$s2, \$s3</code>
<code>slti</code>	I	10	18	17		100		<code>slti \$s1, \$s2, 100</code>
<code>sltu</code>	R	0	18	19	17	0	43	<code>sltu \$s1, \$s2, \$s3</code>
<code>sltiu</code>	I	11	18	17		100		<code>sltiu \$s1, \$s2, 100</code>
Field size		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	R	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	I	op	rs	rt	constant or address			Data transfer format

Procedures (subroutines) الإجراءات

جميع اللغات عالية المستوى تدعم الإجراءات وهي عبارة عن برنامج فرعي أو تابع ينفذ مهمة محددة، يُستدعى عند الحاجة لإعادة استخدام الـ code وتجنب التكرار، وقد يأخذ parameters أو يعود بقيمة لكن ذلك ليس شرطاً أساسياً وهي عبارة عن برنامج فرعي مخزن سابقاً ينفذ مهمة محددة بناءً على الـ parameters يجعل فهم البرنامج أسهل ويسمح بإعادة استخدام الـ code

```
int main(void)
{
    int A, B, C, D, avg;
    avg = average(A, B, C, D);
    return 0;
}
```

مثال :

Procedures in Assembly lang

يمر المعالج ب 6 خطوات متتالية عند استدعاء أي دالة:

- 1 - تمرير المعاملات: وضع الباراميترات في مسجلات خاصة لتقرأها الدالة
- 2 - نقل التحكم: القفز الى عنوان الدالة في الذاكرة
- 3- حجز مساحة: أي مكان بالذاكرة الـ stack للمتغيرات المحلية للدالة
- 4- التنفيذ : القيام بالعمليات الحسابية او المنطقية المطلوبة
- 5- وضع قيمة الـ return في مسجلات خاصة ليراها الـ caller
- 6- الرجوع: العودة الى نقطة الاستدعاء الأساسية لمتابعة البرنامج

ملاحظة: caller هو المستدعي و callee هو المستدعى

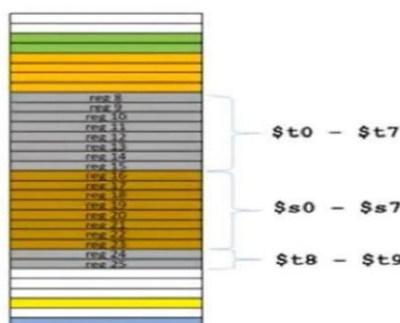
```
calle: average(int a, int b, int c, int d)
{
    return (a+b+c+d)/4;
}
```

```
Caller: int main()
{
    avg=average(A,B,C,D);
}
```

Procedure call: important registers

Register 0	\$zero
Register 1	
Register 2	\$v0
Register 3	\$v1
Register 4	\$a0
Register 5	\$a1
Register 6	\$a2
Register 7	\$a3
Register 8	
Register 9	
...	
Register 29	\$sp
Register 30	
Register 31	\$ra

تأمل المخططات الهندسية التالية التي توضح هيكلية وتوزع السجلات الـ 32 داخل المعالج:



اسم السجل في الكود	أرقام السجلات الوظيفية الأكاديمية (حسب السلايدات)	
\$v0 - \$v1	2-3	إعادة قيم النتائج (Return Values) تُستخدم لتقييم وإعادة النتائج من الإجراء المستدعى إلى البرنامج الرئيسي.
\$a0 - \$a3	4-7	تمرير المعاملات (Arguments): تُستخدم لنقل المعطيات والوسائط من البرنامج الرئيسي إلى الإجراء.
\$t0 - \$t9	15 و 24-25	سجلات مؤقتة (Temporary Registers): سجلات غير محمية؛ لا يتم الحفاظ على قيمها عند استدعاء إجراء آخر (Not preserved across procedure call)
\$s0 - \$s7	16-23	سجلات محفوظة (Saved Registers): سجلات محمية؛ يجب الحفاظ على قيمها الأصلية واستعادتها في حال استخدامها من قبل الإجراء المستدعى استناداً إليها (Must be SAVED and RESTORED).
\$sp	29	مؤشر المكس (Stack Pointer): يشير إلى العنوان الأحدث المخصص في ذاكرة المكس.
\$ra	31	سجل عنوان العودة (Return Address): يقوم بحفظ عنوان التعليمة التالية في البرنامج الرئيسي للعودة إليها بعد انتهاء الإجراء.



Procedure call: instructions

نحن بحاجة دائما الى سجل للاحتفاظ بعناوين التعليمات الحالية التي يتم تنفيذها يسمى عادة `program counter` ويختصر `pc`

عند قراءة كود ما ظهر فجأة استدعاء لتابع هنا يوجد للمعالج مهمتين:

1 تنفيذ الدالة المستدعاة 2 تذكر مكان وقوفه بالضبط في البرنامج الرئيسي للعودة عند انتهاء تنفيذ الدالة

Procedure call: jump and link

التعليمة: `jal procedureAddress`

الوظيفة:

1 تضع عنوان التعليمة التالية في `$ra` حيث تنظر الى مكان وقوف ال `PC` و تزيد عليه 4 للوصول الى السطر الذي يلي عملية الاستدعاء وتحفظ هذا العنوان في السجل `$ra` (سجل عنوان العودة)

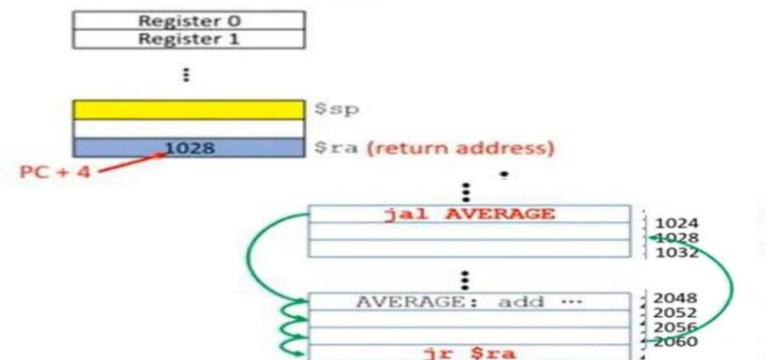
2 تقفز الى عنوان الهدف `TA -> target address` حيث يتم تحديث العداد: `PC -> TA`

Procedure return: jump register

التعليمة: `jr $ra`

الوظيفة: تنسخ القيمة الموجودة في `$ra` الى `PC`

حيث تكتب في آخر سطر داخل الدالة مما يؤمن عودة المعالج الى البرنامج الرئيسي بسلام



Example: Compiling Procedure (average)

If saved registers are used, the callee must assume that the caller needs their values

• Recall procedure to compute average between four scores:

```
int average(int a, int b, int c, int d)
{
    int average_score;
    average_score = (a + b + c + d) / 4;
    return average_score;
}
```

• Corresponding MIPS assembly code:

```
AVERAGE:  add    $s0, $a0, $a1
            add    $s0, $s0, $a2
            add    $s0, $s0, $a3
            srl    $s0, $s0, 2
            add    $v0, $s0, $zero
            jr     $ra
```

parameters

result

Problem: previous contents of `$s0` erased!!

المكدس واستدعاء الاجراءات (Procedure call: The stack)

عند القيام بكتابة برامج, فاننا نحتاج احيانا لاستدعاء (اجراءات) او دوال. أين ستحفظ هذه البيانات المؤقتة أثناء الاستدعاء؟

هنا يأتي دور المكدس : أولا ماهو المكدس؟ وهو بنية بيانات تعمل بمبدأ (الذي يدخل اخيرا يخرج اولاً) طبعا سنستخدمه لتفريغ السجلات وحفظ محتوياتها مؤقتا لكي لا تضيع البيانات
تم تخصيص السجل رقم 29 ليكون مؤشر المكدس، ويحفظ عنوان قمة المكدس
وينمو المكدس من العناوين العليا نزولا الى العناوين الدنيا وتحديدًا من العنوان 7FFFFFFc
ولنضع بيانات في المكدس نستخدم العملية push وللازالة هذا العنصر نستخدم pop

مثال(حساب المتوسط):

- الوضع القديم (غير الآمن):
يقوم بحساب المعدل مباشرة داخل السجل المحفوظ \$s0 مما يتسبب في مسح القيمة القديمة التي كتن البرنامج الرئيسي يحتاجها في هذت السجل دون حفظها.

```
Avarage:      add    $s0, $a0, $a1
               add    $s0, $s0, $a2
               add    $s0, $s0, $a3
               srl    $s0, $s0, 2
               add    $v0, $s0, $zero
               jr     $ra
```

- عند تنفيذ اجراء (procedure) مثل حساب المتوسط, فإننا نحتاج لاستخدام السجل \$s0 لكي لا نفقد قيمته السابقة فنقوم بحفظه في ال stack هكذا:

```
addi $sp,$sp,-4
```

نطرح 4 من مؤشر المكدس لتفريغ مساحة

```
sw $s0,0($sp)
```

نحفظ قيمة السجل في المكدس

```
add $s0, $a0, $a1
add $s0, $s0, $a2
add $s0, $s0, $a3
srl $s0, $s0, 2
add $v0, $s0, $zero
```

```
lw $s0,0($sp)
```

بعد انتهاء الاجراء نستعيد القيمة

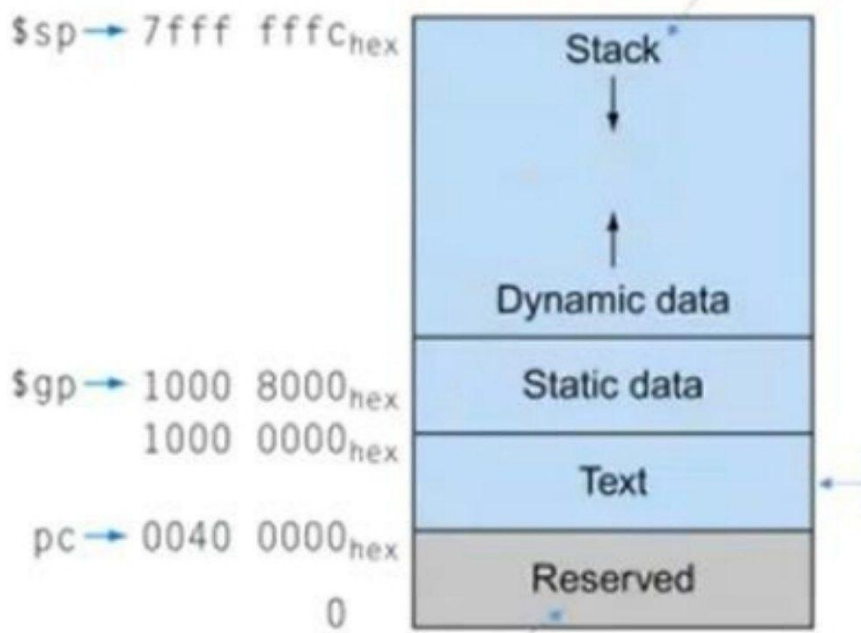
```
addi $sp,$sp,4
```

ثم نعيد مؤشر ال stack لرد المساحة

```
Jr $ra
```


تخطيط الذاكرة في MIPS (Memory Layout)

الذاكرة في برامجنا ليست عشوائية بل مقسمة بشكل هندسي دقيق
 النص (Text): هنا يخزن كود البرنامج (التعليمات) ويبدأ من العنوان 0040 0000
 البيانات الثابتة (Static data): تخزن المتغيرات العامة global التي تراها كل الاجراءات في البرنامج.
 نصل اليها باستخدام السجل \$gp (مؤشر المتغيرات العامة) الذي يهيا عند العنوان 1000 8000
 مثال: static vars in c, constant Arrays and strings.
 البيانات الديناميكية (Heap): تستخدم للبيانات التي يتم حجزها اثناء تشغيل البرنامج (مثل دوال new في java) وتنمو على عكس الستاك أي للأعلى
 مثال: Malloc un c.
 المكديس (Stack): يبدأ من اعلى الذاكرة 7fff fffc وينمو نزولا باتجاه ال heap كما انه يستخدم
 لحفظ المتغيرات المحلية local مثل المصفوفات
 الجزء المحجوز (Reserved): جزء مخصص لنظام التشغيل ولا يمكن للبرنامج استخدامه مباشرة
 ويستخدم ايضا للمقاطعات (interrupts) ولعمليات الادخال/الاخراج



تعليمات التفرع واتخاذ القرار (Branch Instruction)

لكي يكون الحاسوب ذكيا، يجب ان يتخذ القرارات وهنا سنستخدم تعليمات التفرع التي تغير قيمة مسجل البرنامج (PC) الى موقع قريب. تستخدم برمجيا لبناء الجمل الشرطية if else والحلقات Loops.
 هذه التعليمات هي من نوع I-type :

opcode	rs	rt	16-bit signed constant
--------	----	----	------------------------

Opcode: تحديد هل التعليمة beq bne

rs and rt: تحديد الرجسترات التي يتم المقارنة بينها

16-bit signed constant: قيمة ثابتة تمثل العنوان الفوري المستهدف للقفز

- تعليمة beq(Branch if equal): تتفرع إذا كان السجلان متساويان
- تعليمة bne(Branch not equal): تتفرع إذا كان السجلان غير متساويان

إذا كيف يُحسب العنوان الجديد؟

- إذا تطابق الشرط $(PC+4) + 4 \times \text{sxt}(\text{immediate})$
- إذا لم يتحقق الشرط: $PC+4$

لماذا نضرب ب 4؟ لأن كل تعليمة في MIPS تشغل 4 بايت حيث قيمة القفزة المكتوبة بالكود تمثل عدد الاسطر ولكي يحولها الم عالج الى عنوان حقيقي بالبايت يضربها ب 4 ثم يضيفها لعنوان التعليمة التالية ليقفز الى السطر الجديد فوراً

العمليات الحسابية الفورية و التعامل مع الثوابت الكبيرة

التعليمات الفورية (Immediate) تضع الرقم مباشرة داخل التعليمة مثلاً تعليمة `addi $21,$22,-50` تُترجم الى لغة الالة كالتالي:

$Rs=22$ (السجل المصدر)

$Rt=21$ (السجل الهدف)

$\text{Immediate}=-50$

ماذا لو كان الثابت كبير يعني مثلاً بيوصل لل 32 bit ؟

نعلم ان حقل ال immediate يتسع ل 16 بت فقط، لذا سنستخدم تسلسلاً من تعليمتين

- تعليمة التحميل العلوي الفوري `lui`: تضع النصف الاول (ال 16 بت اليسرى) في السجل

- تعليمة المعاملة المنطقية `ori`: تضع النصف الثاني (ال 16 بت اليمنى) لتكتمل القيمة

`lui $8,1010101010101010`

`ori $8,$8,1010101010101010`

الرموز والسلاسل النصية (Characters and String)

كيف يقرأ الحاسوب النص؟ نعلم ان السلسلة النصية هي مجرد مصفوفة من الحروف

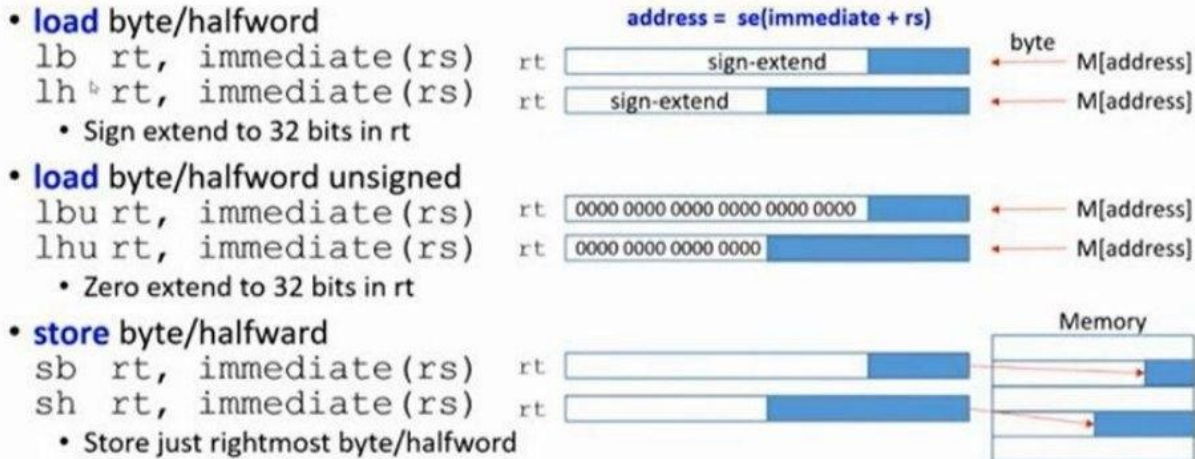
- نظام ASCII: يمثل الحرف في 8 بت (بايت واحد)
- نظام Unicode: يمثل الحرف في 16 بت (UTF-16 وهو الافتراضي) او 32 بت (UTF-32) ليغطي كل لغات العالم

كيف بنحدد نهاية هي السلسلة النصية؟ في طريقتين يا اما نحجز موضع الاول لتسجيل طول السلسلة، او الطريقة الاشهر (كما في لغة c) وهي وضع حرف فارغ (`NULL` (\0)) في النهاية لعلامة انتهاء النص

تحميل و تخزين البايتات (Bytes and Halfwords)

بما ان الحروف تمثل بالبايتات فان MIPS يوجد به تعليمات مخصصة لها

- lh و lb : تحميل بايت او نصف كلمة مع توسيع الاشارة الى 32 بت لملء السجل
- lhu و lbu : تحميل بدون اشارة وتملاً باقي السجل بالأصفار
- sh و sb : تخزين بايت او نصف كلمة حيث يتم اخذ الجزء الايمن فقط من السجل وتخزينه في الذاكرة



Full credit of these slides is given to Drs. Mohammad Sinky and AbdulBasit Abid

78

معمارية التحميل و التخزين (LOAD/STORE Architecture)

ملاحظة: في MIPS العمليات الحسابية و المنطقية (ALU) تتم حصراً داخل السجلات و الوسيلة الوحيدة للتواصل مع الذاكرة هي عبر تعليمات Load و Store

مثال: `lw $t0, 32($s2)` وتعني هذه التعليمة: اذهب الى العنوان الموجود في السجل الاساسي `$s2` اضع اليه الازاحة 32 بايت واجلب القيمة لتضعها في `$t0` وهذه التعليمة هي من نوع I-type .

. نسخ سلسلة نصية (strcpy)

لنفترض اننا نكتب دالة C تنسخ السلسلة y الى x. العناوين الاساسية للمصفوفتين موجودة في السجلات `$a0` و `$a1` والعداد i سيكون في `$s0`

strcpy:

`addi $sp, $sp, -4` # حجز ال Stack لعنصر واحد

`sw $s0, 0($sp)` # تخزين `$s0`

`add $s0, $zero, $zero` # `i=0`

`add $t1, $s0, $a1` # حساب عنوان `y[i]` في `$t1`



```

void stcspy(char[] x,char[] y)
{
    int i=0;
    while((x(i)=y[i])!='\0')
    {
        i++;
    }
}

```

\$t2=y[i] # lbu \$t2, 0(\$t1)
 \$t3 add \$t3, \$s0, \$a0 #حساب عنوان x[i] في \$t3
 x[i]=y[i] # sb \$t2, 0(\$t3)
 y[i]==0 beq \$t2, \$zero, L2 #الخروج من الحلقة اذا كان
 i=i+1 , addi \$s0, \$s0, 1 #زيادة العداد
 # الانتقال للتكرار التالي للحلقة L1
 L2: lw \$s0, 0(\$sp) #استعادة قيمة السجل \$s0 المحفوظة
 pop # addi \$sp,\$sp, 4 أي اخراج عنصر واحد من ال stack

ملاحظة امتحانية: لا نضرب العداد ب 4 عند حساب العناوين لأن حجم حرف ال char في الذاكرة هو 1 byte فقط وليس Integer

تحسين الأداء: يمكن الاستغناء عن ال stack لتسريع الكود اذا استخدمنا سجلا مؤقتا مثل \$t0 كعداد بدلا من السجل المحفوظ \$s0

شرط النهاية: التعليمة beq \$t2, \$zero, L2 تفحص حرف النهاية الصفري (Null) و بمجرد الوصول اليه يتم الخروج من الحلقة لانهاء الدالة

MIPS addressing طرق العنونة وآلية التعامل مع العناوين والثوابت

32-bit immediates and address-1

كيف تحمل رقم ثابت حجمه 32 بت كامل بالمسجل؟

بالوضع الطبيعي المعالج بكفيه 16 بت للثوابت الصغيرة ولكن عند وجود ثابت كبير 32 بت لا يمكن أن يتسع في

تعليلة واحدة لذلك تقسم العملية لخطوتين : مثلاً نريد تخزين 32 بت في \$s0

lui \$s0 , constantUpper

بما أن حجمه كبير نأخذ النصف العلوي فقط منه (أول 16 بت) وتضعه في النصف العلوي للسجل \$s0 والنصف الآخر

من \$s0 يتم تصفيره لتجهيزه للعملية التالية

ori \$s0,\$s0,constantLower

تم دمج النصفين العلوي والسفلي حتى وصلنا لسجل كامل 32 بت

Your monitor deserves a break from your face, u can take a break 🤖

مثال: خزن الثابت : 0000 0000 1001 0000 1101 0000 1101 0011 0000 0000

61

2304

Lui \$s0 , 61

Ori \$s0 , \$s0 , 2304

Conditional branch addressing-2

التعليمات الشرطية في MIPS تحدد opcode وسجلين ويتبقى 16 بت مخصصة للعنوان الذي نريد القفز إليه (target address) تتواجد التعليمات الشرطية في الحلقات loops و if statement

البرامج بطبيعتها عند وجود حلقة أو شرط الكود عادةً يقفز إلى سطر قريب إما للأمام forward مثلاً في حال عدم تحقق الشرط ... أو للخلف backward في حال تكرار الحلقة

المشكلة : عناوين الذاكرة في MIPS حجمها 32 بت بينما التعليمات الشرطية لا تخصص سوى 16 بت للعنوان

الحل: pc-relative addressing المعالج لا يحفظ العنوان الحقيقي في الـ 16 بت بل يحفظ المسافة (عدد الخطوات offset) بالنسبة لمكانه الحالي المسجل بالـ pc

النقطة المرجعية للحساب (Base address) : لا يبدأ المعالج حساب العنوان من تعليمة الشرط نفسها بل يبدأ من التعليمة التي تليها مباشرة في الذاكرة والتي تساوي $pc+4$

وحدة قياس المسافة : يقوم معالج MIPS بتوسيع مدى مسافة القفز المتاحة (stretches the distance) بالاعتماد على عدد الكلمات (#of words) بدلاً من عدد البايتات وتسمى هذه العملية برمجياً بـ word-aligning

$$Target\ address = (pc + 4) + offset \times 4$$

والـ offset هي المسافة اللازمة للوصول للموقع الهدف

The range : مدى القفز المسموح هو أقصى مسافة يمكن لتعليمة الشرط الوصول لها سواء للأمام أو للخلف وتحسب بالمتراجحة:

$$pc + 4 - 4 \times 2^{15} \leq address \leq pc + 4 + 4 \times (2^{15} - 1)$$

op	address
6 bits	26 bits

3. Jump Addressing

تستخدم تعليمات القفز غير المشروط مثل j و jal للانتقال إلى أي موقع داخل (Text Segment) في الذاكرة، وتعتمد على الصيغة J-Format حيث يتم ترميز العنوان الكامل بـ 26 بت:

Op=6 bits	Address=26 bits
-----------	-----------------

طريقة حساب عنوان الهدف للقفز وتسمى Pseudo-Direct Jump Addressing بما أن العناوين في MIPS تحتاج 32 بت والتعليمة لا تملك سوى 26 بت يتم تحويل الـ 26 بت إلى عنوان 32 بت كامل عبر الخطوات:

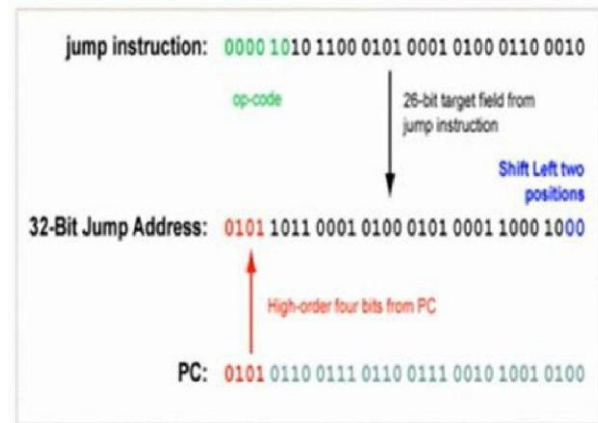
1. Word aligning : يتم إزاحة اليسار بمقدار خانتين (لـ 26 بت) أي ضربها بـ 4 مما يضيف صفرين 00 في الخانات الدنيا low-order فتتحول إلى 28 بت

2. إكمال العنوان (High-order bits) : يتم أخذ الخانات العليا الأربع من عداد البرنامج الحالي pc ووضعها في أعلى العنوان المراد تشكيله ... النتيجة : دمج البتات الأربع من pc مع الـ 28 بت الناتجة يعطينا العنوان النهائي بـ 32 بت.

القانون الرياضي لحساب عنوان الهدف للقفز :

$$Target\ address = pc_{31...28} : (address \times 4)$$

حيث تعني النقطتان عملة الدمج أو الـ concatenation



مثال:

النتيجة : (Implication) تعليمة القفز مقيدة بالحركة داخل بلوك واحد في الذاكرة حجمه 256 ميغابايت 2^{28} لذلك يجب على برامج ربط وتحميل الكود في الذاكرة (linker & loader) تجنب وضع البرنامج على الحدود الفاصلة بين البلوكات

Target Address Example

• Loop code from earlier example

• Assume Loop at location 80000

```
Loop: sll $t1, $s3, 2
      add $t1, $t1, $s6
      lw  $t0, 0($t1)
      bne $t0, $s5, Exit
      addi $s3, $s3, 1
      j   Loop
Exit: ...
```

- Loops also rely on conditional branches
- while (save[i] == k) i += 1;
- Mapping i to \$s3 and k to \$s5
- Base of array save is \$s6 (save[0] = \$s6)

0	0	19	9	2	0
0	9	22	9	0	32
35	9	8		0	
5	8	21		2	
8	19	19		1	
2					20000

Target Address Example

• Loop code from earlier example

• Assume Loop at location 80000

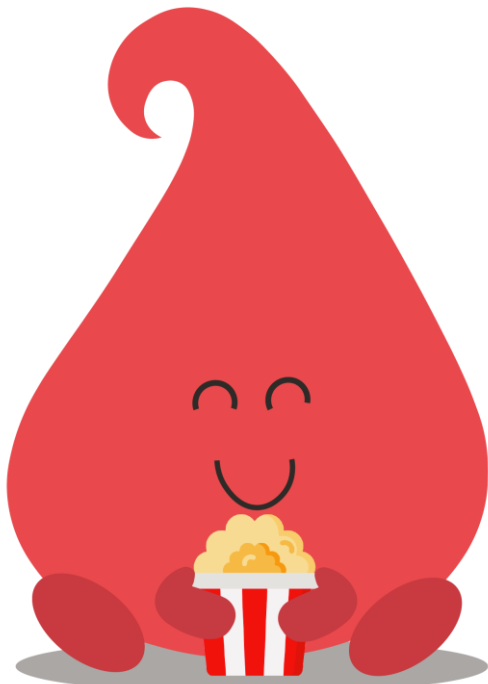
```
Loop: sll $t1, $s3, 2
      add $t1, $t1, $s6
      lw  $t0, 0($t1)
      bne $t0, $s5, Exit
      addi $s3, $s3, 1
      j   Loop
Exit: ...
```

Uses relative address

Uses full address

- Loops also rely on conditional branches
- while (save[i] == k) i += 1;
- Mapping i to \$s3 and k to \$s5
- Base of array save is \$s6 (save[0] = \$s6)

80000	0	0	19	9	2	0	R
80004	0	9	22	9	0	32	R
80008	35	9	8		0		I
80012	5	8	21		2		I
80016	8	19	19		1		I
80020	2					20000	J
80024							



Branching far away : في حال كان التفرع بعيد المدى ليتم ترميزه بـ 16 بت assembler بعيد كتابة الكود

Beq \$s0,\$s1,l1



Bne \$s0,\$s1,L1
J L1
L2:

خلاصة انماط العنونة (Summary Addressing Modes)

هناك خمسة طرق للوصول للبيانات في MIPS :

- العنونة الفورية (immediate): المعامل هو ثابت موجود داخل التعليمة نفسها
- عنونة السجل (Register): المعامل موجود داخل السجل
- العنونة القاعدية/اللازحة (Base/displacement): العنوان في الذاكرة = قيمة سجل + ثابت (اللازحة)
- العنونة المرتبطة بعدد البرنامج (PC-relative): تستخدم للتفرع العنوان $PC +$ (الثابت مضروب بأربعة من أجل محاذاة الكلمة)
- تنويه: ال PC المذكور هنا يقصد به هندسيا عنوان التعليمة التالية $PC+4$ عند حساب المسائل
- العنونة شبه المباشرة (Pseudodirect): تستخدم لتعليمات القفز ز و al نأخذ ال 26 بت من التعليمة نضربها ب 4 و ندمجها مع أعلى أربعة بتات من ال PC الحالي لإنتاج عنوان 32 بت
- الضرب ب 4 يعني عمليا إزاحة خانتين نحو اليسار 2 shift left لتتحول ال 26 بت إلى 28 بت

ملاحظة: لو كان التفرع بعيد جدا أي أن الهدف أبعد من أن يستوعبه حقل ال 16 بت في التفرع العادي فيقوم المجمع بعكس الشرط (مثل تحويل beq إلى bne) ثم يضع تعليمة قفز غير مشروطة ز للوصول للهدف البعيد

مبادئ التصميم

من أجل تصميم معمارية RISC قوية مثل MIPS التزم المهندسون بهذه المبادئ :

- البساطة تفضل الانتظام (Simplicity favors regularity)
- الأصغر هو الأسرع (Smaller is faster)
- اجعل الحالة الشائعة سريعة (Make the common case fast)
- التصميم الجيد يتطلب تنازلات جيدة (Good design demands good compromises)

تذكير بطبقات hardware/software التي يمر بها البرنامج ليتنفذ:

compiler -1
assembler -2
hardware- 3

MIPS هي المثال الأشهر والاقوى ل RISC ISAs

The tears in your prayers
are the ink Allah uses to
write MIRACLES !!

